XX 服装订单交易平台 V1.0

软

件

源

代

码

节

选

XX

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace WindowsFormsApplication1
{
    static class Program
    {
        /// <summary>
        ///
        /// </summary>
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new Form1());
        }
    }
}
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

namespace WebApplication3.Models
{
    public class ExternalLoginConfirmationViewModel
    {
        [Required]
        [Display(Name = "       ")]
        public string Email { get; set; }
    }

    public class ExternalLoginListViewModel
    {
        public string ReturnUrl { get; set; }
    }

    public class SendCodeViewModel
    {
        public string SelectedProvider { get; set; }
        public ICollection<System.Web.Mvc.SelectListItem> Providers { get; set; }
        public string ReturnUrl { get; set; }
        public bool RememberMe { get; set; }
    }

    public class VerifyCodeViewModel
    {
        [Required]
        public string Provider { get; set; }

        [Required]
        [Display(Name = "    ")]
        public string Code { get; set; }
```

```
        public string ReturnUrl { get; set; }

        [Display(Name = "          ?")]
        public bool RememberBrowser { get; set; }

        public bool RememberMe { get; set; }
    }

    public class ForgotViewModel
    {
        [Required]
        [Display(Name = "       ")]
        public string Email { get; set; }
    }

    public class LoginViewModel
    {
        [Required]
        [Display(Name = "       ")]
        [EmailAddress]
        public string Email { get; set; }

        [Required]
        [DataType(DataType.Password)]
        [Display(Name = "    ")]
        public string Password { get; set; }

        [Display(Name = "     ?")]
        public bool RememberMe { get; set; }
    }

    public class RegisterViewModel
    {
        [Required]
        [EmailAddress]
        [Display(Name = "       ")]
        public string Email { get; set; }

        [Required]
         [StringLength(100, ErrorMessage = "{0}             {2}         ", MinimumLength =
6)]
        [DataType(DataType.Password)]
        [Display(Name = "    ")]
        public string Password { get; set; }

        [DataType(DataType.Password)]
        [Display(Name = "       ")]
        [Compare("Password", ErrorMessage = "                 ")]
        public string ConfirmPassword { get; set; }
    }

    public class ResetPasswordViewModel
    {
        [Required]
        [EmailAddress]
```

```
            [Display(Name = "       ")]
            public string Email { get; set; }

            [Required]
             [StringLength(100, ErrorMessage = "{0}              {2}         ", MinimumLength =
6)]
            [DataType(DataType.Password)]
            [Display(Name = "    ")]
            public string Password { get; set; }

            [DataType(DataType.Password)]
            [Display(Name = "       ")]
            [Compare("Password", ErrorMessage = "                  ")]
            public string ConfirmPassword { get; set; }

            public string Code { get; set; }
        }

        public class ForgotPasswordViewModel
        {
            [Required]
            [EmailAddress]
            [Display(Name = "       ")]
            public string Email { get; set; }
        }
}
using System;
using System.Globalization;
using System.Linq;
using System.Security.Claims;
using System.Threading.Tasks;
using System.Web;
using System.Web.Mvc;
using Microsoft.AspNet.Identity;
using Microsoft.AspNet.Identity.Owin;
using Microsoft.Owin.Security;
using WebApplication3.Models;

namespace WebApplication3.Controllers
{
    [Authorize]
    public class AccountController : Controller
    {
        private ApplicationSignInManager _signInManager;
        private ApplicationUserManager _userManager;

        public AccountController()
        {
        }

      publicAccountController(ApplicationUserManageruserManager,ApplicationSignInManager
signInManager )
        {
            UserManager = userManager;
            SignInManager = signInManager;
```

```
        }

        public ApplicationSignInManager SignInManager
        {
            get
            {
                return _signInManager ?? HttpContext.GetOwinContext().Get<ApplicationSignInManager>();
            }
            private set
            {
                _signInManager = value;
            }
        }

        public ApplicationUserManager UserManager
        {
            get
            {
                return _userManager ?? HttpContext.GetOwinContext().GetUserManager<ApplicationUserManager>();
            }
            private set
            {
                _userManager = value;
            }
        }

        //
        // GET: /Account/Login
        [AllowAnonymous]
        public ActionResult Login(string returnUrl)
        {
            ViewBag.ReturnUrl = returnUrl;
            return View();
        }

        //
        // POST: /Account/Login
        [HttpPost]
        [AllowAnonymous]
        [ValidateAntiForgeryToken]
        public async Task<ActionResult> Login(LoginViewModel model, string returnUrl)
        {
            if (!ModelState.IsValid)
            {
                return View(model);
            }

            //
            // shouldLockout: true
            var result = await SignInManager.PasswordSignInAsync(model.Email, model.Password, model.RememberMe, shouldLockout: false);
            switch (result)
            {
```

```
                    case SignInStatus.Success:
                        return RedirectToLocal(returnUrl);
                    case SignInStatus.LockedOut:
                        return View("Lockout");
                    case SignInStatus.RequiresVerification:
                  return RedirectToAction("SendCode", new { ReturnUrl = returnUrl, RememberMe =
model.RememberMe });
                    case SignInStatus.Failure:
                    default:
                        ModelState.AddModelError("",  "                ");
                        return View(model);
                }
            }

            //
            // GET: /Account/VerifyCode
            [AllowAnonymous]
              public async Task<ActionResult> VerifyCode(string provider, string returnUrl, bool
rememberMe)
            {
                //                    /
                if (!await SignInManager.HasBeenVerifiedAsync())
                {
                    return View("Error");
                }
               return View(new VerifyCodeViewModel { Provider = provider, ReturnUrl = returnUrl,
RememberMe = rememberMe });
            }

            //
            // POST: /Account/VerifyCode
            [HttpPost]
            [AllowAnonymous]
            [ValidateAntiForgeryToken]
            public async Task<ActionResult> VerifyCode(VerifyCodeViewModel model)
            {
                if (!ModelState.IsValid)
                {
                    return View(model);
                }

                //
                //
                //
                //        IdentityConfig
              var result = await SignInManager.TwoFactorSignInAsync(model.Provider, model.Code,
isPersistent:    model.RememberMe, rememberBrowser: model.RememberBrowser);
                switch (result)
                {
                    case SignInStatus.Success:
                        return RedirectToLocal(model.ReturnUrl);
                    case SignInStatus.LockedOut:
                        return View("Lockout");
                    case SignInStatus.Failure:
                    default:
```

```
                    ModelState.AddModelError("",  "          ");
                    return View(model);
                }
            }

            //
            // GET: /Account/Register
            [AllowAnonymous]
            public ActionResult Register()
            {
                return View();
            }

            //
            // POST: /Account/Register
            [HttpPost]
            [AllowAnonymous]
            [ValidateAntiForgeryToken]
            public async Task<ActionResult> Register(RegisterViewModel model)
            {
                if (ModelState.IsValid)
                {
                    var user = new ApplicationUser { UserName = model.Email, Email =
model.Email };
                    var result = await UserManager.CreateAsync(user, model.Password);
                    if (result.Succeeded)
                    {
                        await SignInManager.SignInAsync(user, isPersistent:false,
rememberBrowser:false);

                        //                    http://go.microsoft.com/fwlink/?
LinkID=320771
                        //
                        // string code = await
UserManager.GenerateEmailConfirmationTokenAsync(user.Id);
                        // var callbackUrl = Url.Action("ConfirmEmail", "Account", new { userId = user.Id,
code = code }, protocol: Request.Url.Scheme);
                        // await UserManager.SendEmailAsync(user.Id, "          ", "          <a
href=\"" + callbackUrl + "\">   </a>          ");

                        return RedirectToAction("Index", "Home");
                    }
                    AddErrors(result);
                }

                //
                return View(model);
            }

            //
            // GET: /Account/ConfirmEmail
            [AllowAnonymous]
            public async Task<ActionResult> ConfirmEmail(string userId, string code)
            {
                if (userId == null || code == null)
```

```
            {
                return View("Error");
            }
            var result = await UserManager.ConfirmEmailAsync(userId, code);
            return View(result.Succeeded ? "ConfirmEmail" : "Error");
        }

        //
        // GET: /Account/ForgotPassword
        [AllowAnonymous]
        public ActionResult ForgotPassword()
        {
            return View();
        }

        //
        // POST: /Account/ForgotPassword
        [HttpPost]
        [AllowAnonymous]
        [ValidateAntiForgeryToken]
        public async Task<ActionResult> ForgotPassword(ForgotPasswordViewModel model)
        {
            if (ModelState.IsValid)
            {
                var user = await UserManager.FindByNameAsync(model.Email);
                if (user == null || !(await UserManager.IsEmailConfirmedAsync(user.Id)))
                {
                    //
                    return View("ForgotPasswordConfirmation");
                }

                    //                                                  http://go.microsoft.com/fwlink/?
LinkID=320771
                //
                                                                    // string code = await
UserManager.GeneratePasswordResetTokenAsync(user.Id);
                // var callbackUrl = Url.Action("ResetPassword", "Account", new { userId = user.Id,
code = code }, protocol: Request.Url.Scheme);
                    // await UserManager.SendEmailAsync(user.Id, "        ", "            <a
href=\"" + callbackUrl + "\">  </a>            ");
                // return RedirectToAction("ForgotPasswordConfirmation", "Account");
            }

            //
            return View(model);
        }

        //
        // GET: /Account/ForgotPasswordConfirmation
        [AllowAnonymous]
        public ActionResult ForgotPasswordConfirmation()
        {
            return View();
        }
```

```
        //
        // GET: /Account/ResetPassword
        [AllowAnonymous]
        public ActionResult ResetPassword(string code)
        {
            return code == null ? View("Error") : View();
        }

        //
        // POST: /Account/ResetPassword
        [HttpPost]
        [AllowAnonymous]
        [ValidateAntiForgeryToken]
        public async Task<ActionResult> ResetPassword(ResetPasswordViewModel model)
        {
            if (!ModelState.IsValid)
            {
                return View(model);
            }
            var user = await UserManager.FindByNameAsync(model.Email);
            if (user == null)
            {
                //
                return RedirectToAction("ResetPasswordConfirmation", "Account");
            }
                var result = await UserManager.ResetPasswordAsync(user.Id, model.Code,
model.Password);
            if (result.Succeeded)
            {
                return RedirectToAction("ResetPasswordConfirmation", "Account");
            }
            AddErrors(result);
            return View();
        }

        //
        // GET: /Account/ResetPasswordConfirmation
        [AllowAnonymous]
        public ActionResult ResetPasswordConfirmation()
        {
            return View();
        }

        //
        // POST: /Account/ExternalLogin
        [HttpPost]
        [AllowAnonymous]
        [ValidateAntiForgeryToken]
        public ActionResult ExternalLogin(string provider, string returnUrl)
        {
            //
          return new ChallengeResult(provider, Url.Action("ExternalLoginCallback", "Account",
new { ReturnUrl = returnUrl }));
        }
```

```
        //
        // GET: /Account/SendCode
        [AllowAnonymous]
        public async Task<ActionResult> SendCode(string returnUrl, bool rememberMe)
        {
            var userId = await SignInManager.GetVerifiedUserIdAsync();
            if (userId == null)
            {
                return View("Error");
            }
            var userFactors = await UserManager.GetValidTwoFactorProvidersAsync(userId);
           var factorOptions = userFactors.Select(purpose => new SelectListItem { Text = purpose,
Value = purpose }).ToList();
             return View(new SendCodeViewModel { Providers = factorOptions, ReturnUrl =
returnUrl, RememberMe = rememberMe });
        }

        //
        // POST: /Account/SendCode
        [HttpPost]
        [AllowAnonymous]
        [ValidateAntiForgeryToken]
        public async Task<ActionResult> SendCode(SendCodeViewModel model)
        {
            if (!ModelState.IsValid)
            {
                return View();
            }

            //
            if (!await SignInManager.SendTwoFactorCodeAsync(model.SelectedProvider))
            {
                return View("Error");
            }
             return RedirectToAction("VerifyCode", new { Provider = model.SelectedProvider,
ReturnUrl = model.ReturnUrl, RememberMe = model.RememberMe });
        }

        //
        // GET: /Account/ExternalLoginCallback
        [AllowAnonymous]
        public async Task<ActionResult> ExternalLoginCallback(string returnUrl)
        {
            var loginInfo = await AuthenticationManager.GetExternalLoginInfoAsync();
            if (loginInfo == null)
            {
                return RedirectToAction("Login");
            }

            //
                var result = await SignInManager.ExternalSignInAsync(loginInfo, isPersistent:
false);
            switch (result)
            {
                case SignInStatus.Success:
```

```
                    return RedirectToLocal(returnUrl);
                case SignInStatus.LockedOut:
                    return View("Lockout");
                case SignInStatus.RequiresVerification:
                return RedirectToAction("SendCode", new { ReturnUrl = returnUrl, RememberMe =
false });
                case SignInStatus.Failure:
                default:
                    //
                    ViewBag.ReturnUrl = returnUrl;
                    ViewBag.LoginProvider = loginInfo.Login.LoginProvider;
                                        return View("ExternalLoginConfirmation", new
ExternalLoginConfirmationViewModel { Email = loginInfo.Email });
            }
        }

        //
        // POST: /Account/ExternalLoginConfirmation
        [HttpPost]
        [AllowAnonymous]
        [ValidateAntiForgeryToken]
                                        public async Task<ActionResult>
ExternalLoginConfirmation(ExternalLoginConfirmationViewModel model, string returnUrl)
        {
            if (User.Identity.IsAuthenticated)
            {
                return RedirectToAction("Index", "Manage");
            }

            if (ModelState.IsValid)
            {
                //
                var info = await AuthenticationManager.GetExternalLoginInfoAsync();
                if (info == null)
                {
                    return View("ExternalLoginFailure");
                }
                    var user = new ApplicationUser { UserName = model.Email, Email =
model.Email };
                var result = await UserManager.CreateAsync(user);
                if (result.Succeeded)
                {
                    result = await UserManager.AddLoginAsync(user.Id, info.Login);
                    if (result.Succeeded)
                    {
                  await SignInManager.SignInAsync(user, isPersistent: false, rememberBrowser:
false);
                        return RedirectToLocal(returnUrl);
                    }
                }
                AddErrors(result);
            }

            ViewBag.ReturnUrl = returnUrl;
            return View(model);
```

```
}

//
// POST: /Account/LogOff
[HttpPost]
[ValidateAntiForgeryToken]
public ActionResult LogOff()
{
    AuthenticationManager.SignOut(DefaultAuthenticationTypes.ApplicationCookie);
    return RedirectToAction("Index", "Home");
}

//
// GET: /Account/ExternalLoginFailure
[AllowAnonymous]
public ActionResult ExternalLoginFailure()
{
    return View();
}

protected override void Dispose(bool disposing)
{
    if (disposing)
    {
        if (_userManager != null)
        {
            _userManager.Dispose();
            _userManager = null;
        }

        if (_signInManager != null)
        {
            _signInManager.Dispose();
            _signInManager = null;
        }
    }

    base.Dispose(disposing);
}

#region
//                      XSRF
private const string XsrfKey = "XsrfId";

private IAuthenticationManager AuthenticationManager
{
    get
    {
        return HttpContext.GetOwinContext().Authentication;
    }
}

private void AddErrors(IdentityResult result)
{
    foreach (var error in result.Errors)
```

```
            {
                ModelState.AddModelError("", error);
            }
        }

        private ActionResult RedirectToLocal(string returnUrl)
        {
            if (Url.IsLocalUrl(returnUrl))
            {
                return Redirect(returnUrl);
            }
            return RedirectToAction("Index", "Home");
        }

        internal class ChallengeResult : HttpUnauthorizedResult
        {
            public ChallengeResult(string provider, string redirectUri)
                : this(provider, redirectUri, null)
            {
            }

            public ChallengeResult(string provider, string redirectUri, string userId)
            {
                LoginProvider = provider;
                RedirectUri = redirectUri;
                UserId = userId;
            }

            public string LoginProvider { get; set; }
            public string RedirectUri { get; set; }
            public string UserId { get; set; }

            public override void ExecuteResult(ControllerContext context)
            {
                var properties = new AuthenticationProperties { RedirectUri = RedirectUri };
                if (UserId != null)
                {
                    properties.Dictionary[XsrfKey] = UserId;
                }
                  context.HttpContext.GetOwinContext().Authentication.Challenge(properties,
LoginProvider);
            }
        }
        #endregion
    }
}
```